

Multilingual Constituency Parsing with Self-Attention and Pre-Training

Yusuf Fakhri Aldrian*
Fakultas Ilmu Komputer
Universitas Indonesia
Depok, Indonesia
Email: *yusuf.fakhri@ui.ac.id

Abstract—Penelitian paper ini dilakukan untuk mereplikasi hasil eksperimen N. Kitaev et.al. yang mengembangkan constituency parser multibahasa dimana penulis sendiri menggunakan bahasa Inggris dan juga bahasa Indonesia. Untuk penelitian bahasa Inggris, penulis tidak mencapai nilai hasil eksperimen dari paper acuan. Sedangkan untuk penelitian bahasa Indonesia, penulis mendapatkan bahwa nilai hasil eksperimen yang dihasilkan IndoBERT lebih tinggi dibandingkan mBERT yang membuktikan hasil eksperimen yang didapatkan dari paper acuan. Untuk penelitian berikutnya, penulis menyarankan untuk peneliti berikutnya untuk mencoba berbagai jenis pre-training dari paper N. Kitaev et.al. untuk bahasa Indonesia.

Index Terms—constituency parsing, bert, self attention encoder

I. PENDAHULUAN

Constituency parsing merupakan suatu proses penguraian kalimat berdasarkan konstituen penyusunnya yaitu frasa. Masing-masing konstituen yang telah diurai tersebut diberi label yang menandakan peran dari konstituen tersebut di dalam suatu kalimat.

Terdapat dua macam label yang digunakan, yakni POS dan syntactic tag. POS (Part-of-Speech) tag merupakan label yang diasosiasikan pada satu buah token/part-of-speech, seperti DT: Determiner, VB: Verb, dsb. Sementara itu, syntactic tag merupakan label yang disematkan pada sebuah frasa/konstituen, seperti VP: Verb Phrase, NP: Noun Phrase, PP: Prepositional Phrase, dsb.

Constituency tree T direpresentasikan sebagai kumpulan labeled span, dimana Parser memberikan skor $s(T)$ untuk setiap tree. Untuk setiap span

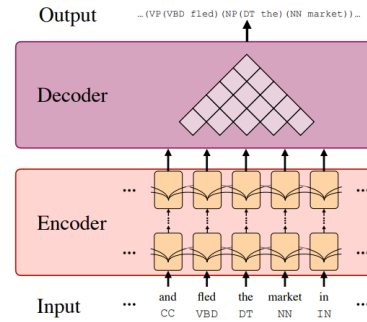


Fig. 1. Arsitektur parser menggabungkan decoder chart dengan encoder kalimat berdasarkan self-attention.

dalam kalimat, vektor dibuat dengan mengurangi representasi yang terkait dengan awal dan akhir span. Sebuah klasifikasi span MLP, yang terdiri dari dua lapisan sepenuhnya terhubung dengan satu non-linearitas ReLU, memberikan skor label $s(i, j, \cdot)$ untuk span tersebut.

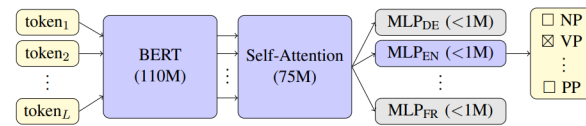


Fig. 2. Arsitektur model multilingual.

Paper ini menyoroti manfaat pre-training secara unsupervised dalam konteks berbagai bahasa. Melalui perbandingan performance, paper menunjukkan bahwa BERT unggul terutama karena ka-

pasitas model yang lebih besar yang juga berlaku untuk 11 bahasa yang diuji. Disini, peneliti mengeksplorasi model multibahasa dan fine-tuning, memungkinkan berbagi parameter antara sepuluh bahasa dalam satu model. Meskipun terdapat pengurangan ukuran model hingga 10 kali lipat, hanya terjadi peningkatan error relatif sebesar 3.2%. Selanjutnya, paper mengusulkan ide fine-tuning bersama untuk memberikan bahasa dengan resource rendah ke dataset yang lebih besar dari bahasa lain. Hasil akhir dari paper ini adalah state-of-the-art untuk 11 bahasa saat paper ditulis.

Untuk riset yang dilakukan penulis sendiri, penulis mencoba mereproduksi percobaan yang dilakukan oleh paper acuan dengan beberapa dataset. Penulis melakukan percobaan dengan dataset bahasa Inggris dan dataset bahasa Indonesia yang berasal dari ICON. Untuk word embedding, penulis menggunakan English BERT (bert-base-uncased), Multilingual BERT (bert-base-multilingual-uncased), dan IndoLEM (indolem/indobert-base-uncased).

II. CARA KERJA PROGRAM

A. Penjelasan fungsi train secara general

Program di bawah ini adalah skrip untuk melatih model neural network dalam tugas penguraian tree. Program memuat kumpulan tree pelatihan dan pengembangan dari jalur file tertentu, dengan opsi untuk memfilter contoh berdasarkan panjang maksimum yang ditentukan dalam hyperparameter. Berikut adalah potongan fungsi train yang dipakai untuk training dataset.

```
hparams.set_from_args(args)
print("Hyperparameters:")
hparams.print()

print("Loading training trees from
{ }...".format(args.train_path))
train_treebank = treebanks.load_trees(
    args.train_path, args.
    train_path_text, args.
    text_processing
)
if hparams.max_len_train > 0:
    train_treebank = train_treebank.
    filter_by_length(hparams.
    max_len_train)
```

```
print("Loaded { }, training examples.".
    format(len(train_treebank)))

print("Loading development trees from
{ }...".format(args.dev_path))
dev_treebank = treebanks.load_trees(
    args.dev_path, args.dev_path_text,
    args.text_processing
)
if hparams.max_len_dev > 0:
    dev_treebank = dev_treebank.
    filter_by_length(hparams.
    max_len_dev)
print("Loaded { }, development examples
.".format(len(dev_treebank)))

print("Constructing vocabularies...")
label_vocab = decode_chart.ChartDecoder.
    build_vocab(train_treebank.trees)
if hparams.use_chars_lstm:
    char_vocab = char_lstm.
    RetokenizerForCharLSTM.
    build_vocab(train_treebank.
    sents)
else:
    char_vocab = None

tag_vocab = set()
for tree in train_treebank.trees:
    for _, tag in tree.pos():
        tag_vocab.add(tag)
tag_vocab = ["UNK"] + sorted(tag_vocab)
tag_vocab = {label: i for i, label in
    enumerate(tag_vocab)}

if hparams.force_root_constituent.lower()
    in ("true", "yes", "1"):
    hparams.force_root_constituent =
    True
elif hparams.force_root_constituent.
    lower() in ("false", "no", "0"):
    hparams.force_root_constituent =
    False
elif hparams.force_root_constituent.
    lower() == "auto":
    hparams.force_root_constituent = (
        decode_chart.ChartDecoder.
        infer_force_root_constituent
        (train_treebank.trees)
    )
print("Set hparams.
    force_root_constituent to",
    hparams.force_root_constituent)

print("Initializing model...")
parser = parse_chart.ChartParser(
    tag_vocab=tag_vocab,
    label_vocab=label_vocab,
    char_vocab=char_vocab,
    hparams=hparams,
)
```

```

if args.parallelize:
    parser.parallelize()
elif torch.cuda.is_available():
    parser.cuda()
else:
    print("Not using CUDA!")

```

Listing 1. Fungsi train pada main.py.

Selanjutnya, skrip membuat kosakata untuk label, karakter, dan tag POS. Model ini diinisialisasi menggunakan hyperparameter yang ditentukan. Kemudian, Adam optimizer diinisialisasi untuk melatih model, dengan parameter seperti learning speed, beta, dan epsilon. Skrip kemudian memasuki loop training, mengulangi setiap epoch dan batch. Loop ini menghitung kerugian, melakukan propagasi mundur, dan memperbarui parameter model.

Secara berkala, skrip mengevaluasi model pada set pengembangan, menghitung F-score, dan menyimpan model jika mencapai F-score terbaik sejauh ini. Learning rate disesuaikan berdasarkan scheduler. Training akan dihentikan jika tidak ada peningkatan dalam pengembangan F-score untuk sejumlah decay berturut-turut. Secara keseluruhan, skrip ini dirancang untuk melatih parser neural network, memantau kinerjanya pada dataset development, dan menyimpan model yang merupakan chart parser untuk syntactic tree parsing.

B. Cara kerja ChartParser

1) *Model neural network properti f-label Chart-Parser:* Mari kita fokuskan perhatian ke bagian parser yang terdapat pada snip kode di atas.

```

print("Initializing model...")
parser = parse_chart.ChartParser(
    tag_vocab=tag_vocab,
    label_vocab=label_vocab,
    char_vocab=char_vocab,
    hparams=hparams,
)

```

Listing 2. Bagian parser dalam fungsi train.

Deklarasi tersebut memanggil kelas ChartParser yang terdapat di dalam folder benepar. Pada line 110 hingga 115 terdapat deklarasi seperti berikut.

```
self.f_label = nn.Sequential(
```

```

nn.Linear(hparams.d_model, hparams.
    d_label_hidden),
nn.LayerNorm(hparams.d_label_hidden
    ),
nn.ReLU(),
nn.Linear(hparams.d_label_hidden,
    max(label_vocab.values()))),
)

```

Listing 3. Bagian parser dalam fungsi train.

Arsitektur neural network pada property `self.f_label` dapat dijelaskan sebagai berikut.

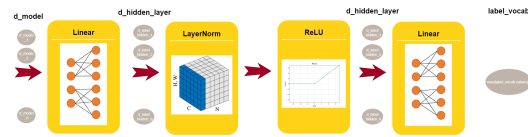


Fig. 3. Diagram fungsi aktivasi ReLU.

Pertama, terdapat input layer dengan ukuran sebesar `hparams.d_model`, yang menerima input data. Input ini kemudian dilewatkan ke linear layer pertama, yang merupakan layer transformasi linear dengan ukuran input sebesar `hparams.d_model` dan ukuran output sebesar `hparams.d_label_hidden`. Fungsi dari linear layer ini adalah melakukan transformasi linear pada input.

Selanjutnya, output dari linear layer pertama melewati layer normalisasi (`LayerNorm`) dengan ukuran input sebesar `hparams.d_label_hidden`. Layer normalisasi ini berfungsi untuk meningkatkan stabilitas dan kecepatan konvergensi model dengan menormalisasi nilai-nilai input. Setelah normalisasi, output dilewatkan melalui fungsi aktivasi ReLU (Rectified Linear Unit), yang memperkenalkan non-linearitas pada model.

Langkah terakhir adalah output dari ReLU diarahkan ke linear layer kedua (output layer). Linear layer kedua ini memiliki ukuran input sebesar `hparams.d_label_hidden` dan ukuran output sebesar `max(label_vocab.values())`. Fungsi dari linear layer ini adalah menghasilkan output final dengan dimensi sesuai dengan jumlah kategori dalam `label_vocab`.

Secara keseluruhan, arsitektur ini menciptakan dataflow dari input ke output melalui beberapa layer

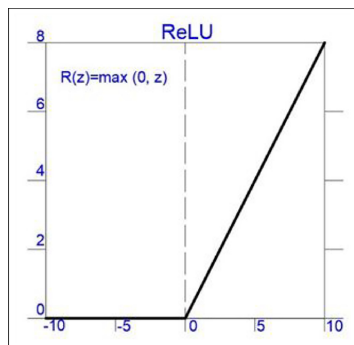


Fig. 4. Diagram fungsi aktivasi ReLU.

transformasi dan normalisasi. Setiap layer memiliki parameter yang dapat diatur, dan fungsi aktivasi ReLU diterapkan untuk memperkenalkan non-linearitas pada model. Semoga penjelasan ini memberikan gambaran yang lebih jelas tentang arsitektur neural network yang digunakan pada `property self.f_label`.

2) *Pemuatan model*: Berikut adalah potongan kode yang ada pada `benepar/parse_chart.py`.

```
parser = cls(**config)
parser.load_state_dict(state_dict)
```

Listing 4. Kode pemuatan model pada `benepar/parse_chart.py`.

Pada kode ini, `config` merujuk pada konfigurasi model yang disimpan. Konfigurasi ini berisi informasi-informasi seperti arsitektur model, hyperparameters, dan setelan lain yang diperlukan untuk membangun kembali model dengan struktur yang sama seperti saat itu di-*train*.

Sementara itu, `state_dict` adalah representasi parameter-parameter model yang telah dilatih. Variabel `state_dict` mencakup bobot dan bias dari setiap layer dalam model. Dengan kata lain, `state_dict` dapat digunakan untuk menginisialisasi model baru dengan parameter yang sama.

Pada dasarnya, potongan kode ini digunakan untuk membuat objek model dengan memanggil constructor `cls` dengan konfigurasi dan kemudian menginisialisasi model tersebut dengan parameter-parameter yang sesuai yang disimpan dalam state dict.

III. HASIL UJI COBA DAN ANALISIS

Berikut adalah tabel evaluasi hasil train dan test pada dataset

TABLE I
HASIL EKSPERIMEN: PTB DAN ICON

No.	Model	Layer	Batch size	Batch no.	LR	LP	F1 score
1	PTB enBERT	2	32	Epoch 1 623/1,245	87.56	88.60	88.08
2	PTB mBERT	2	32	Epoch 1 312/1,245	81.50	78.91	80.18
3	ICON mBERT	2	32	Epoch 3 63/250	82.72	83.54	83.13
4	ICON IndoLEM	2	32	Epoch 3 63/250	82.95	83.85	83.40

A. Constituency Parsing Bahasa Inggris

Untuk eksperimen constituency parsing bahasa Indonesia, sudah disediakan dataset yang diberikan Nikita Kitaev. Treebank tersebut berbentuk format Penn Treebank dan memiliki format file CLEAN.

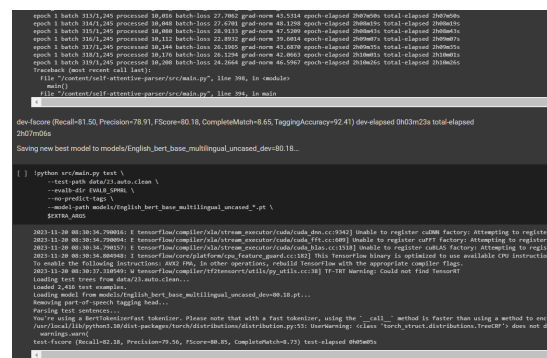


Fig. 5. Kode dan hasil testing multilingual BERT (mBERT).

Recall, Precision, dan F1 Score adalah metrik evaluasi yang umum digunakan dalam pemrosesan NLP dan tugas klasifikasi lainnya. Recall bertugas mengukur sejauh mana model dapat mengidentifikasi semua contoh positif yang sebenarnya dalam dataset. Lalu, precision mengukur sejauh mana prediksi positif model adalah benar. Terakhir, F1 score menggabungkan recall dan precision untuk digunakan sebagai parameter ukuran hasil eksperimen. Hasil eksperimen menunjukkan enBERT memberikan hasil parametrik yang lebih tinggi dibanding

mBERT. Ini disebabkan karena enBERT didesain khusus untuk bahasa Inggris, sedangkan mBERT menghitung banyak bahasa.

Seperti yang terlihat diatas, waktu untuk menjalankan training mencapai 2 jam. Untuk model yang dihasilkan, ukuran file tersebut antara 1,3GB-1,8GB. Ukuran ini terbilang cukup besar karena ada banyak parameter didalamnya.

B. Constituency Parsing Bahasa Indonesia

Untuk eksperimen constituency parsing bahasa Indonesia, sudah disediakan dua dataset yaitu Kethu dan ICON (Sea Core NLP). Kethu adalah sebuah constituency treebank bahasa Indonesia yang awalnya bermula dari Dinakaramani yang diubah menjadi format Penn Treebank oleh Ika Alfina and Jessica Naraiswari Arwidarasti. Awalnya, penulis hendak menggunakan treebank Kethu untuk eksperimen ini. Namun, terjadi Assertion Error dengan pesan berikut.

```
Traceback (most recent call last):
  File "/content/self-attentive-
    parser/src/benepar/
    decode_chart.py", line 59, in
    _get_labeled_spans
    assert len(tree) > 1 or
    isinstance(
AssertionError: Must call
collapse_unary_strip_pos first
```

Listing 5. Assertion Error saat Training Kethu

Sea Core NLP atau yang bisa disebut sebagai ICON adalah utilitas NLP berbahasa Indonesia yang dibentuk oleh perusahaan Singapura Sea. Untuk eksperimen ini, penulis memakai dataset ICON yang diolah dengan IndoLEM IndoBERT dan multilanguage BERT seperti yang digunakan untuk bahasa Inggris.

IndoBERT adalah pre-trained model yang dilatih menggunakan Transformer, algoritma yang diadaptasi dari cara kerja Convolutional Neural Network, hanya saja feature extraction pada Transformer tidak dilakukan dengan cara berkonvolusi menggunakan kernel $n \times m$ seperti pada CNN, namun melibatkan encoder dan decoder. Untuk benchmarking Indo-BERT ini, digunakan MBERT (Multilingual

BERT), MalayBERT (khusus bahasa Malay), dan Bi-LSTM-CRF. Didapat dari penelitian F. Koto, dari hasil comparative evaluationnya, IndoBERT mengungguli hampir semua task. [1]

Fig. 6. Kode dan hasil training IndoBERT.

Dibanding dengan eksperimen bahasa Inggris yang dilakukan sebelumnya, banyak batch dalam satu kali epoch adalah 250. Menurut penulis, ini terjadi karena dataset treebank bahasa Indonesia lebih kecil daripada treebank bahasa Inggris. Karena itu, penulis menjalankan training sampai 3 epoch.

Berdasarkan tabel 1, hasil eksperimen untuk IndoLEM IndoBERT lebih baik daripada mBERT. Ini bisa dilihat dari jumlah Recall, Precision dan F1 score yang lebih tinggi dari mBERT. Ini didasarkan dari IndoBERT yang didesain khusus untuk bahasa Indonesia dan juga ini membuktikan hasil penelitian F. Koto.

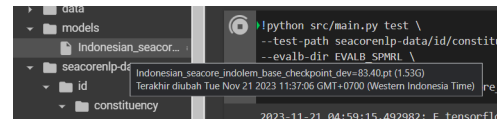


Fig. 7. Hasil model IndoBERT yang dijalankan pada ICON.

IV. KESIMPULAN DAN SARAN

Pada dasarnya, penelitian ini dilakukan untuk mereplikasi hasil eksperimen N. Kitaev et.al. yang mengembangkan constituency parser multibahasa yang memungkinkan peneliti melatih model dengan mengumpulkan seluruh bahasa yang dikehendaki dalam satu dokumen dengan tujuan meningkatkan efisiensi model. Disini, penulis mereplikasi penelitian tersebut untuk bahasa Inggris dan juga bahasa Indonesia.

Untuk penelitian bahasa Inggris, penulis tidak mencapai hasil eksperimen N. Kitaev et.al. dari

paper acuan. Menurut penulis, ini terjadi karena waktu training yang dilakukan pada eksperimen penulis tidak selama waktu training yang dilakukan N. Kitaev et.al. yang disebabkan keterbatasan waktu yang dialokasikan Google Colab. Sedangkan untuk penelitian bahasa Indonesia, penulis membandingkan F-score yang dihasilkan training dengan IndoLEM IndoBERT dan multilingual BERT. Penulis mendapatkan bahwa nilai hasil eksperimen yang dihasilkan IndoBERT lebih tinggi dibandingkan mBERT yang membuktikan hasil eksperimen yang didapatkan dari paper F. Koto [1].

Untuk penelitian berikutnya, penulis menyarankan untuk peneliti berikutnya untuk mencoba berbagai jenis pre-training yang dilakukan N. Kitaev et.al untuk pre-training bahasa Indonesia. Peneliti berikutnya disarankan untuk membandingkan penggunaan BERT, ELMo, fastText dan dengan parser yang ditrain sendiri untuk bahasa Indonesia.

ACKNOWLEDGMENT

Saya berterima kasih kepada dosen saya, Bu Ika Alfina dan asisten dosen saya, Muhammad Faisal Adi Soesatyo yang telah membimbing saya dalam mata kuliah ini dan dalam membantu saya menyelesaikan paper ini. Saya juga berterima kasih kepada Nikita Kitaev, et al. yang telah menyediakan paper mereka yaitu *IndoLEM and IndoBERT: A benchmark dataset and pre-trained language model for Indonesian NLP* [1], *Constituency Parsing with a Self-Attentive Encoder* [2], dan *Multilingual Constituency Parsing with Self-Attention and Pre-Training* [3] sebagai acuan topik paper saya.

REFERENCES

- [1] F. Koto, A. Rahimi, J. H. Lau, and T. Baldwin, "IndoLEM and IndoBERT: A benchmark dataset and pre-trained language model for Indonesian NLP," in *Proceedings of the 28th International Conference on Computational Linguistics*, D. Scott, N. Bel, and C. Zong, Eds. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020, pp. 757–770. [Online]. Available: <https://aclanthology.org/2020.coling-main.66>
- [2] N. Kitaev and D. Klein, "Constituency parsing with a self-attentive encoder," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 2676–2686. [Online]. Available: <https://www.aclweb.org/anthology/P18-1249>
- [3] N. Kitaev, S. Cao, and D. Klein, "Multilingual constituency parsing with self-attention and pre-training," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3499–3505. [Online]. Available: <https://www.aclweb.org/anthology/P19-1340>